Interview Preparation Guide

JavaScript · Playwright · CI/CD · Azure DevOps

100+ Real-World Q&A with Code Examples & References – Ranjit Appukutti

Covers: ES6+ | Async/Await | Promises | Playwright Architecture | POM | Network Mocking
Fixtures | Visual Testing | API Testing | Azure Pipelines | YAML | Parallel Execution
Test Data Management | Accessibility Testing | Best Practices & Anti-Patterns

Section 1: JavaScript Core Concepts

Foundational JavaScript questions essential for automation engineers.

Q1 **What is the difference between var, let, and const in JavaScript?**

ANS

var is function-scoped and hoisted to the top of its scope, where it is initialized with undefined. It allows both re-declaration and re-assignment.
let and const are block-scoped and are also hoisted, but they remain in the **Temporal Dead Zone (TDZ)** until their declaration is executed, meaning they cannot be accessed before initialization.
const must be initialized at declaration and its binding cannot be reassigned, although properties of objects or arrays declared with const can still be mutated.
In automation frameworks, it is recommended to use **const for configuration values and constants**, and **let for variables that change during execution**, such as loop counters or temporary test data.

Code Example:

```
// var - function scoped, hoisted
var count = 1;

// let - block scoped, re-assignable
let timeout = 5000;
timeout = 10000; // OK

// const - block scoped, no re-assignment
const BASE_URL = 'https://app.example.com';
```

```
const config = { browser: 'chromium' };
config.headless = true; // OK - mutating object is allowed
```

Ref: *MDN Web Docs - var/let/const, ECMAScript 2015 Spec §13.3*

Q2 What are Promises and how do they work in JavaScript?

ANS

A Promise represents an asynchronous operation that will complete in the future, returning either a resolved value or a rejection reason. Promises transition from 'pending' to either 'fulfilled' or 'rejected'. They chain with .then(), .catch(), and .finally(). In Playwright, almost all APIs return Promises, making this knowledge critical.

Code Example:

```
// Creating a Promise
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => resolve('data loaded'), 1000);
});

// Consuming
fetchData
  .then(data => console.log(data))
  .catch(err => console.error(err))
  .finally(() => console.log('done'));
```

Ref: *MDN - Using Promises, ECMAScript 2015 §25.4*

Q3 Explain async/await and how it simplifies Promise handling.

ANS

async/await provides a cleaner and more readable way to work with Promises.
An async function always returns a Promise. The await keyword pauses execution **within the async function** until the Promise resolves or rejects, while the JavaScript event loop continues processing other tasks.
This makes asynchronous code easier to read and maintain compared to chained .then() calls. In Playwright, most browser interactions return Promises, which is why async/await is commonly used in test scripts.

Code Example:

```
// Without async/await (messy chaining)
page.goto('https://example.com')
  .then(() => page.click('#btn'))
  .then(() => page.waitForSelector('.modal'));

// With async/await (clean and readable)
```

```
async function runTest() {
  await page.goto('https://example.com');
  await page.click('#btn');
  await page.waitForSelector('.modal');
}
```

Ref: *MDN - async function, TC39 Async Functions Proposal*

Q4 What is closure in JavaScript and why is it important in test automation?

ANS

A closure is a function that retains access to its lexical scope even when the function is executed outside that scope. In test automation, closures are used in fixtures, helper factories, and configuration callbacks. They allow test utilities to 'remember' configuration without global variables.

Code Example:

```
// Factory function using closure
function createApiHelper(baseUrl, token) {
  // baseUrl and token are 'closed over'
  return async function request(endpoint) {
    const res = await fetch(`${baseUrl}${endpoint}`, {
      headers: { Authorization: `Bearer ${token}` }
    });
    return res.json();
  };
}

const api = createApiHelper('https://api.example.com', 'secret');
const users = await api('/users'); // baseUrl & token remembered
```

Ref: *You Don't Know JS - Scope & Closures (Kyle Simpson)*

Q5 What is the difference between == and === in JavaScript?

ANS

== performs abstract equality with type coercion, meaning it converts operands to the same type before comparison. === performs strict equality without coercion, comparing both value and type. In test automation, always use === to avoid unexpected false positives in assertions.

Code Example:

```
// Loose equality - type coercion
0 == false   // true  ← dangerous in tests!
'' == false  // true
null == undefined // true
```

```
// Strict equality - no coercion
0 === false  // false ✓ safer
'' === false // false ✓

// In test assertions
expect(response.status).toBe(200); // uses ===
```

📖 **Ref:** *ECMAScript Spec §7.2.12, MDN - Equality comparisons*

Q6 What are arrow functions and how do they differ from regular functions?

ANS Arrow functions (=>) have a shorter syntax and do not bind their own 'this', 'arguments', or 'super'. They inherit 'this' from the enclosing lexical scope. In Playwright tests, arrow functions are typically used for test callbacks and page.evaluate() calls.

Code Example:

```
// Regular function - has its own 'this'
const obj = {
  name: 'Test',
  run: function() { return this.name; } // 'Test'
};

// Arrow function - inherits 'this'
const arrowObj = {
  name: 'Test',
  run: () => this.name // undefined (lexical 'this')
};

// In Playwright - arrow functions for evaluate
const title = await page.evaluate(() => document.title);
```

📖 **Ref:** *MDN - Arrow function expressions, TC39 ES2015*

Q7 Explain destructuring assignment in JavaScript with automation examples.

ANS Destructuring allows unpacking values from arrays or properties from objects into distinct variables. It's commonly used in Playwright to extract test context fixtures, configuration values, and API responses.

Code Example:

```
// Object destructuring
const { chromium, firefox, webkit } = require('@playwright/test');
```

```
// Array destructuring
const [firstItem, secondItem] = await page.$$('.list-item');

// Nested destructuring from API response
const { data: { user: { name, email } } } = await api.getUser(1);

// Default values + renaming
const { timeout = 30000, retries: maxRetries = 3 } = config;
```

Ref: *MDN - Destructuring assignment, Playwright Docs - Fixtures*

Q8 What is the event loop in JavaScript and how does it relate to test execution?

ANS

The event loop is JavaScript's mechanism for handling asynchronous operations. It has a call stack (sync code), a heap (memory), and queues (microtasks like Promises, macrotasks like setTimeout). Understanding it is critical for knowing why tests should use await and not block the main thread.

Code Example:

```
// Microtask queue (Promises) runs before macrotask (setTimeout)
console.log('1');
Promise.resolve().then(() => console.log('2 - microtask'));
setTimeout(() => console.log('3 - macrotask'), 0);
console.log('4');
// Output: 1, 4, 2, 3

// Why this matters in Playwright
await page.waitForSelector('#modal'); // Promise - correct
// Never use blocking loops - they starve the event loop
```

Ref: *What the heck is the event loop? - Philip Roberts (JSConf EU 2014)*

Q9 What are JavaScript modules (ESM vs CommonJS)?

ANS

CommonJS (require/module.exports) is Node.js's original module system, loaded synchronously. ESM (import/export) is the standard JavaScript module system, supports static analysis and tree-shaking. Playwright supports both but its own packages are primarily ESM-compatible.

Code Example:

```
// CommonJS (older, Node.js default)
const { test } = require('@playwright/test');
module.exports = { baseURL: 'https://example.com' };
```

```
// ESM (modern, preferred)
import { test, expect } from '@playwright/test';
export default { baseURL: 'https://example.com' };

// package.json
{ "type": "module" } // enables ESM
```

📖 **Ref:** *Node.js Docs - Modules, Playwright Docs - Configuration*

Q10 What is optional chaining (?.) and nullish coalescing (??) in JavaScript?

ANS

Optional chaining (?.) safely accesses deeply nested properties without throwing if an intermediate value is null or undefined. Nullish coalescing (??) returns the right-hand value only if the left-hand side is null or undefined (unlike ||, which also triggers on empty string and 0). Both are heavily used in test data setup.

Code Example:

```
// Optional chaining - safe deep access
const city = user?.address?.city; // undefined instead of TypeError
const firstResult = results?.[0]?.title;
const name = obj?.getName?.(); // safe method call

// Nullish coalescing
const timeout = config.timeout ?? 30000; // use default only if null/undefined
const retries = config.retries ?? 2;

// Combined in automation
const baseURL = process.env.BASE_URL ?? 'https://staging.example.com';
```

📖 **Ref:** *MDN - Optional chaining, Nullish coalescing (ES2020)*

🎭 Section 2: Playwright – Core Concepts

Playwright fundamentals every QA automation engineer must know.

Q11 What is Playwright and how does it differ from Selenium?

ANS

Playwright is a modern end-to-end testing framework developed by Microsoft that supports Chromium, Firefox, and WebKit browsers.
Unlike Selenium, which communicates with browsers using the **WebDriver protocol and separate browser drivers**, Playwright communicates directly with browser

engines using browser-specific automation protocols. This results in faster execution and more reliable automation.
Playwright also provides built-in capabilities such as:

- Automatic waiting for elements
- Multiple browser contexts for session isolation
- Built-in network interception
- Native parallel execution
- Integrated tracing and debugging tools

These features significantly reduce test flakiness and simplify test development compared to traditional Selenium frameworks.

Code Example:

```
// Playwright - built-in auto-waiting, no explicit waits needed
await page.click('#submit');        // waits for element automatically
await page.fill('#email', 'a@b.c'); // waits for element to be editable

// Selenium (Java) - manual waits often required
WebDriverWait wait = new WebDriverWait(driver, 10);
wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
```

Ref: *Playwright Official Docs - playwright.dev/docs/why-playwright*

Q12 What is a BrowserContext in Playwright and why is it important?

ANS

A BrowserContext is an isolated browser session within a single Browser instance. Each context has its own cookies, local storage, session storage, and cache. This allows running multiple independent test sessions in parallel without interference, like incognito windows. Fixtures in Playwright automatically create a new context per test.

Code Example:

```
// Creating isolated browser contexts
const browser = await chromium.launch();

const context1 = await browser.newContext(); // User A session
const context2 = await browser.newContext(); // User B session

const page1 = await context1.newPage();
const page2 = await context2.newPage();

// Authenticate independently
await page1.goto('/login'); // logs in as admin
await page2.goto('/login'); // logs in as regular user
```

Ref: *Playwright Docs - playwright.dev/docs/browser-contexts*

Q13 How does Playwright's auto-waiting mechanism work?

ANS

Playwright automatically waits for elements to meet certain conditions before performing actions. For click(), it waits for: element visibility, stability (not animating), receives events (not obscured), enabled state, and editable state. This eliminates flaky tests caused by timing issues. The default timeout is 30 seconds (configurable).

Code Example:

```
// Playwright checks ALL of these automatically before click():
// 1. Attached to DOM
// 2. Visible (not display:none or visibility:hidden)
// 3. Stable (not animating/moving)
// 4. Receives pointer events (not covered by overlay)
// 5. Enabled (not disabled attribute)

await page.click('#submit'); // All 5 checks happen automatically

// You can also explicitly wait
await page.waitForSelector('#result', { state: 'visible' });
await page.waitForLoadState('networkidle');
```

Ref: *Playwright Docs - playwright.dev/docs/actionability*

Q14 What are Playwright locators and why are they recommended over page.$?

ANS

Locators are Playwright's preferred way to reference page elements. Unlike page.$() which resolves immediately, locators are lazy and re-query the DOM at each action, making them resilient to DOM updates. They also support strict mode (fail if multiple elements match) and have built-in auto-waiting.

Code Example:

```
// Old approach (page.$) - resolves once, can go stale
const el = await page.$('#submit'); // snapshot at this moment
await el.click(); // may fail if DOM changed

// Modern approach - Locators (recommended)
const submitBtn = page.locator('#submit'); // lazy reference
await submitBtn.click(); // re-queries DOM at click time

// Locator types
page.locator('text=Submit')
page.getByRole('button', { name: 'Submit' })
page.getByLabel('Email')
page.getByTestId('submit-btn')
page.getByPlaceholder('Enter email')
```

Ref: *Playwright Docs - playwright.dev/docs/locators*

Q15 Explain the Page Object Model (POM) in Playwright with a real-world example.

ANS

POM is a design pattern where each web page has a corresponding class that encapsulates its elements and actions. It separates test logic from page implementation, improves maintainability, and reduces duplication. When the UI changes, only the page class needs updating, not every test.

Code Example:

```
// pages/LoginPage.ts
export class LoginPage {
  constructor(private page: Page) {}

  async navigate() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.page.getByLabel('Email').fill(email);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Sign In' }).click();
  }
}

// tests/login.spec.ts
test('successful login', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.navigate();
  await loginPage.login('user@example.com', 'pass123');
  await expect(page).toHaveURL('/dashboard');
});
```

Ref: *Playwright Docs - playwright.dev/docs/pom, Martin Fowler - PageObject*

Q16 How do you handle network interception in Playwright?

ANS

Playwright provides page.route() to intercept, modify, or block network requests. This is useful for mocking API responses, simulating errors, blocking analytics calls, or testing offline scenarios without hitting real backends.

Code Example:

```
// Mock API response
await page.route('**/api/users', async route => {
  await route.fulfill({
    status: 200,
```

```
    contentType: 'application/json',
    body: JSON.stringify([{ id: 1, name: 'Test User' }])
  });
});

// Simulate network error
await page.route('**/api/payment', route => route.abort());

// Modify request headers
await page.route('**/*', route => {
  route.continue({
    headers: { ...route.request().headers(), 'X-Test': 'true' }
  });
});
```

Ref: *Playwright Docs - playwright.dev/docs/network*

Q17 What are Playwright fixtures and how do you create custom ones?

ANS

Fixtures are reusable setup/teardown mechanisms in Playwright. Built-in fixtures include page, browser, context, request. Custom fixtures extend the test object to provide pre-authenticated pages, test data, or shared utilities. They support dependency injection automatically.

Code Example:

```
// fixtures.ts
import { test as base } from '@playwright/test';
import { LoginPage } from './pages/LoginPage';

type MyFixtures = {
  loginPage: LoginPage;
  authenticatedPage: Page;
};

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },

  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.fill('#email', 'admin@test.com');
    await page.fill('#password', 'secret');
    await page.click('#login-btn');
    await use(page); // provide pre-authenticated page
  }
});
```

Ref: *Playwright Docs - playwright.dev/docs/test-fixtures*

Q18 How do you run Playwright tests in parallel and what are isolation strategies?

ANS

Playwright runs tests in parallel by default using worker processes. Each worker gets its own browser context. You can configure parallelism at the test file level, project level, or globally. Workers are isolated processes — they don't share memory, so shared state must use external storage (DB, file, API).

Code Example:

```
// playwright.config.ts
export default defineConfig({
  workers: process.env.CI ? 2 : 4, // CI: fewer workers
  fullyParallel: true, // run tests in files in parallel

  // Per-file: disable parallelism for sequential tests
  // In spec file:
  test.describe.configure({ mode: 'serial' });
});

// Test isolation - each test gets fresh context
test.beforeEach(async ({ context }) => {
  // context is brand new per test
  await context.clearCookies();
});
```

📖 **Ref:** *Playwright Docs - playwright.dev/docs/test-parallel*

Q19 How does Playwright handle authentication and session storage?

ANS

Playwright supports saving authentication state (cookies + localStorage) to a file and reusing it across tests. This avoids logging in before every test, speeding up test suites significantly. Use storageState in globalSetup for one-time auth or in fixtures for role-based auth.

Code Example:

```
// globalSetup.ts - login once, save state
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('https://app.example.com/login');
  await page.fill('#email', process.env.TEST_USER!);
  await page.fill('#password', process.env.TEST_PASS!);
  await page.click('#login-btn');
  await page.context().storageState({ path: 'auth.json' });
  await browser.close();
}
```

```
// playwright.config.ts
export default defineConfig({
  use: { storageState: 'auth.json' } // reuse across all tests
});
```

Ref: *Playwright Docs - playwright.dev/docs/auth*

Q20 What is Playwright's tracing feature and how do you use it for debugging?

ANS Playwright tracing records a full trace of test execution including screenshots, DOM snapshots, network requests, console logs, and action timelines. Traces can be viewed in the Playwright Trace Viewer UI. They are invaluable for debugging failures in CI where you can't observe the browser live.

Code Example:

```
// Enable tracing in playwright.config.ts
use: {
  trace: 'on-first-retry', // only trace on retry (saves space)
  // Options: 'off' | 'on' | 'retain-on-failure' | 'on-first-retry'
}

// Manual tracing in tests
await context.tracing.start({ screenshots: true, snapshots: true });
// ... run test actions ...
await context.tracing.stop({ path: 'trace.zip' });

// View trace
// npx playwright show-trace trace.zip
```

Ref: *Playwright Docs - playwright.dev/docs/trace-viewer*

Q21 How do you perform API testing with Playwright?

ANS Playwright has a built-in APIRequestContext (request fixture) for making HTTP requests without a browser. It shares authentication state with the browser context and supports all HTTP methods, file uploads, and response assertions. Ideal for API setup/teardown and hybrid UI+API tests.

Code Example:

```
// Using the request fixture
test('create and verify user via API', async ({ request, page }) => {
  // Create user via API
  const response = await request.post('/api/users', {
    data: { name: 'John Doe', email: 'john@test.com' }
```

```
  });
  expect(response.status()).toBe(201);
  const user = await response.json();

  // Verify in UI
  await page.goto(`/users/${user.id}`);
  await expect(page.getByText('John Doe')).toBeVisible();

  // Cleanup via API
  await request.delete(`/api/users/${user.id}`);
});
```

Ref: *Playwright Docs - playwright.dev/docs/api-testing*

Q22 Explain visual testing / screenshot comparison in Playwright.

ANS

Playwright supports visual regression testing via toHaveScreenshot() which compares current screenshots against stored baselines. On first run, baselines are created. Subsequent runs detect pixel-level differences. Options include thresholds, masks for dynamic areas, and full-page captures.

Code Example:

```
// Basic screenshot comparison
await expect(page).toHaveScreenshot('homepage.png');

// With options
await expect(page).toHaveScreenshot('dashboard.png', {
  maxDiffPixels: 100,        // allow minor differences
  threshold: 0.2,            // per-pixel color diff threshold
  fullPage: true,            // capture entire page
  mask: [page.locator('.date')] // mask dynamic elements
});

// Element-level screenshot
await expect(page.locator('.chart')).toHaveScreenshot('chart.png');

// Update baselines: npx playwright test --update-snapshots
```

Ref: *Playwright Docs - playwright.dev/docs/screenshots*

Q23 How do you handle iframes in Playwright?

ANS

Playwright interacts with elements inside iframes using the frameLocator() API or by accessing the frame object.
The recommended approach is **frameLocator()**, because it integrates with Playwright's locator system and benefits from auto-waiting and retry logic.

This allows tests to interact with elements inside iframes reliably without needing to manually switch contexts.
Nested iframes can also be handled by chaining frameLocator() calls.

Code Example:

```
// Using frameLocator (recommended - works with locators)
const iframe = page.frameLocator('#payment-iframe');
await iframe.locator('#card-number').fill('4111 1111 1111 1111');
await iframe.locator('#cvv').fill('123');
await iframe.getByRole('button', { name: 'Pay' }).click();

// Using frame() by name or URL
const frame = page.frame({ name: 'checkout-frame' });
const frame2 = page.frame({ url: /stripe.com/ });

// Nested iframes
const nested = page.frameLocator('#outer').frameLocator('#inner');
```

Ref: *Playwright Docs - playwright.dev/docs/frames*

Section 3: Playwright – Advanced Topics

Advanced Playwright topics for senior engineers and architects.

Q24 **What are Playwright projects and how do you use them for cross-browser testing?**

ANS Projects are named configurations in playwright.config.ts that allow running the same tests against different browsers, viewports, or environments. They enable matrix testing across browsers and devices without duplicating test code.

Code Example:

```
// playwright.config.ts
export default defineConfig({
  projects: [
    { name: 'Chrome', use: { ...devices['Desktop Chrome'] } },
    { name: 'Firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'Safari', use: { ...devices['Desktop Safari'] } },
    { name: 'Mobile Chrome', use: { ...devices['Pixel 5'] } },
    { name: 'Mobile Safari', use: { ...devices['iPhone 13'] } },
    // Environment-based project
    {
      name: 'staging',
      use: { baseURL: 'https://staging.example.com' },
      testMatch: /.*.spec.ts/
```

```
    }
  ]
});
```

📖 **Ref:** *Playwright Docs - playwright.dev/docs/test-projects*

Q25 How do you implement data-driven testing in Playwright?

ANS Data-driven testing runs the same test logic with multiple data sets. Playwright supports this through test.describe loops, test.each via loops, or reading from external files (JSON, CSV, Excel). This maximizes coverage while minimizing code duplication.

Code Example:

```
// Using array iteration
const testData = [
  { user: 'admin@test.com', role: 'Admin', canDelete: true },
  { user: 'user@test.com', role: 'User', canDelete: false },
  { user: 'viewer@test.com', role: 'Viewer', canDelete: false },
];

for (const { user, role, canDelete } of testData) {
  test(`${role} permissions test`, async ({ page }) => {
    await loginAs(page, user);
    await expect(page.locator('.role-badge')).toHaveText(role);
    if (canDelete) {
      await expect(page.locator('#delete-btn')).toBeVisible();
    } else {
      await expect(page.locator('#delete-btn')).not.toBeVisible();
    }
  });
}
```

📖 **Ref:** *Playwright Docs - playwright.dev/docs/parameterize-tests*

Q26 How do you handle file uploads and downloads in Playwright?

ANS File uploads use setInputFiles() for input[type=file] elements. For downloads, Playwright provides a download event and waitForEvent('download') to capture the download object, which can then be saved or inspected.

Code Example:

```
// File upload
await page.locator('input[type=file]').setInputFiles('path/to/file.pdf');
```

```
// Multiple files
await page.locator('input[type=file]').setInputFiles([
  'file1.pdf', 'file2.xlsx'
]);

// File download
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.click('#export-btn')        // triggers download
]);
const path = await download.path();
console.log('Downloaded to:', path);

// Verify filename
expect(download.suggestedFilename()).toBe('report-2024.csv');
```

Ref: *Playwright Docs - playwright.dev/docs/downloads*

Q27 How do you test web components and Shadow DOM in Playwright?

ANS

Playwright can interact with elements inside **open Shadow DOM** automatically using standard locators. This allows tests to locate elements inside web components without requiring special handling.
However, **closed Shadow DOM cannot be accessed directly**, because it is intentionally encapsulated by the browser. In such cases, testing must rely on exposed APIs or application-level hooks.

Code Example:

```
// Playwright auto-pierces open shadow DOM
await page.locator('my-component >> input').fill('value');

// Using CSS pierce selector
await page.locator('custom-input >>> input[type=text]').fill('test');

// getByRole and getByLabel also pierce shadow DOM
await page.getByLabel('Username').fill('testuser');

// Check if element is in shadow DOM
const isInShadow = await page.evaluate(() => {
  return document.querySelector('my-widget')
    ?.shadowRoot
    ?.querySelector('button') !== null;
});
```

Ref: *Playwright Docs - playwright.dev/docs/selectors#shadow-piercing-combinators*

Q28 How do you handle dynamic content and wait strategies in Playwright?

ANS

Playwright provides automatic waiting for most actions, which eliminates the need for many manual waits.
However, explicit waiting strategies are available when necessary, such as:

- waitForSelector() – wait for an element state
- waitForURL() – wait for navigation
- waitForLoadState() – wait for page load events
- waitForResponse() – wait for network responses
- waitForFunction() – wait for custom JavaScript conditions

While networkidle can be used to wait until no network requests are active, it should be used carefully. In most cases, it is better to wait for **specific UI conditions or API responses** rather than relying on network inactivity.

Code Example:

```
// Wait for specific element state
await page.waitForSelector('#spinner', { state: 'hidden' });
await page.waitForSelector('.results', { state: 'visible', timeout: 10000 });

// Wait for navigation
await page.waitForURL('**/dashboard');

// Wait for load state
await page.waitForLoadState('networkidle'); // no requests for 500ms

// Wait for API response
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/data')),
  page.click('#load-data-btn')
]);
const data = await response.json();

// Wait for custom condition
await page.waitForFunction(() => document.querySelectorAll('.item').length > 5);
```

Ref: *Playwright Docs - playwright.dev/docs/navigations*

Q29 How do you manage test data and environment configuration in Playwright?

ANS

Best practices: Use .env files with dotenv for secrets, playwright.config.ts for test-level configuration, fixtures for per-test data setup, and test tags for filtering. Never hardcode credentials in test files. Use process.env for environment-specific values and a config abstraction layer.

Code Example:

```
// config/environments.ts
export const ENV = {
  BASE_URL: process.env.BASE_URL || 'https://staging.example.com',
  API_URL: process.env.API_URL || 'https://api-staging.example.com',
  ADMIN_EMAIL: process.env.ADMIN_EMAIL!,
  ADMIN_PASS: process.env.ADMIN_PASS!,
};

// playwright.config.ts
export default defineConfig({
  use: {
    baseURL: ENV.BASE_URL,
    extraHTTPHeaders: {
      'X-Environment': process.env.NODE_ENV || 'test'
    }
  }
});

// .env.staging
BASE_URL=https://staging.example.com
ADMIN_EMAIL=admin@staging.example.com
```

Ref: *Playwright Docs - playwright.dev/docs/test-configuration*

Section 4: CI/CD & Azure DevOps

CI/CD pipeline concepts and Azure DevOps automation practices.

Q30 What is CI/CD and how does it apply to test automation?

ANS

CI (Continuous Integration) is the practice of frequently merging code changes and running automated builds and tests. CD (Continuous Delivery/Deployment) extends CI to automatically deploy verified code to staging or production. In test automation, CI/CD ensures every code change is validated by tests before reaching users.

Code Example:

```
# Simple CI pipeline flow
# 1. Developer pushes code → PR created
# 2. CI triggers: lint → unit tests → build
# 3. Deploy to test environment
# 4. Playwright E2E tests run against test env
# 5. If all pass → merge PR
# 6. CD triggers: deploy to staging → smoke tests
# 7. If pass → deploy to production → sanity tests

# Azure DevOps YAML example
trigger:
  branches:
```

```
    include: [ main, develop ]
  paths:
    exclude: [ '*.md', 'docs/*' ]
```

📖 **Ref:** *Azure DevOps Docs - docs.microsoft.com/en-us/azure/devops/pipelines*

Q31 How do you configure an Azure DevOps pipeline to run Playwright tests?

ANS Create an azure-pipelines.yml file in your repository root. The pipeline installs Node.js, installs dependencies and Playwright browsers, runs tests, and publishes test results and artifacts. Use environment variables from Azure DevOps variable groups for secrets.

Code Example:

```
# azure-pipelines.yml
trigger:
  - main
  - develop

pool:
  vmImage: 'ubuntu-latest'

variables:
  - group: 'playwright-test-vars'  # Variable group with secrets

steps:
  - task: NodeTool@0
    inputs:
      versionSpec: '18.x'

  - script: npm ci
    displayName: 'Install dependencies'

  - script: npx playwright install --with-deps chromium
    displayName: 'Install Playwright browsers'

  - script: npx playwright test --reporter=junit,html
    displayName: 'Run Playwright Tests'
    env:
      BASE_URL: $(BASE_URL)
      ADMIN_PASS: $(ADMIN_PASS)

  - task: PublishTestResults@2
    condition: always()
    inputs:
      testResultsFormat: 'JUnit'
      testResultsFiles: 'test-results/junit.xml'
```

📖 **Ref:** *Azure Pipelines Docs - learn.microsoft.com/en-us/azure/devops/pipelines/tasks*

Q32 What are Azure DevOps variable groups and how do you use them securely?

ANS Variable groups store configuration values and secrets shared across pipelines. Secrets in variable groups are encrypted and not exposed in logs. Link variable groups to pipelines and reference variables using $(variable-name) syntax. For highly sensitive values, integrate with Azure Key Vault.

Code Example:

```
# In azure-pipelines.yml
variables:
  - group: 'playwright-secrets'  # link variable group
  - name: 'NODE_ENV'
    value: 'test'

steps:
  - script: |
      echo "Environment: $(NODE_ENV)"
      # $(SECRET_KEY) is masked in logs as ***
    env:
      SECRET_KEY: $(API_SECRET_KEY)  # from variable group
      DATABASE_URL: $(DB_CONNECTION_STRING)

# Azure Key Vault integration
variables:
  - group: 'keyvault-linked-group'  # linked to Azure Key Vault
  # Secrets are fetched at pipeline runtime
```

Ref: *Azure DevOps Docs - Variable groups & Key Vault integration*

Q33 How do you set up multi-stage pipelines in Azure DevOps?

ANS Multi-stage pipelines define sequential stages like Build, Test, Staging Deploy, Production Deploy. Each stage can have conditions, approvals, and environment gates. This models a real deployment pipeline where artifacts promote through environments.

Code Example:

```
# azure-pipelines.yml - multi-stage
stages:
  - stage: Build
    jobs:
      - job: BuildApp
        steps:
          - script: npm run build
          - task: PublishBuildArtifacts@1

  - stage: Test
    dependsOn: Build
```

```
    jobs:
      - job: E2ETests
        steps:
          - task: DownloadBuildArtifacts@1
          - script: npx playwright test

  - stage: Staging
    dependsOn: Test
    condition: succeeded()
    jobs:
      - deployment: DeployStaging
        environment: 'staging'  # can have approval gates
        strategy:
          runOnce:
            deploy:
              steps:
                - script: ./deploy.sh staging
```

Ref: *Azure Docs - Stages in Azure Pipelines (multi-stage)*

Q34 What are Azure DevOps environments and deployment gates?

ANS

Environments represent logical deployment targets (staging, production) and provide deployment history, approvals, and resource tracking. Deployment gates add automated checks: query Azure Monitor, check work items, invoke Azure Functions, or run REST API checks before allowing deployment to proceed.

Code Example:

```
# azure-pipelines.yml - environment with approval
- stage: Production
  jobs:
    - deployment: DeployProd
      environment:
        name: 'production'  # Configure approvals in Azure DevOps UI
        resourceType: VirtualMachine
      strategy:
        runOnce:
          deploy:
            steps:
              - script: ./deploy-prod.sh

# Environment Gates (configured in UI):
# - Require manual approval from release manager
# - Query: Application Insights error rate < 1%
# - Check: All linked work items are resolved
# - Pre-deployment: Run smoke tests via Azure Function
```

Ref: *Azure DevOps Docs - Deployment gates and approvals*

Q35 How do you publish and view Playwright test reports in Azure DevOps?

ANS

Azure DevOps supports multiple test result formats. Playwright's junit reporter generates JUnit XML which PublishTestResults task displays natively in the pipeline. The HTML report can be published as a build artifact and downloaded. The allure reporter provides the richest reports when combined with Allure Azure extension.

Code Example:

```
# playwright.config.ts
reporter: [
  ['junit', { outputFile: 'test-results/junit.xml' }],
  ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ['list']  // console output
],

# azure-pipelines.yml
- task: PublishTestResults@2
  condition: always()
  displayName: 'Publish Test Results'
  inputs:
    testResultsFormat: 'JUnit'
    testResultsFiles: 'test-results/junit.xml'
    testRunTitle: 'Playwright E2E Tests'
    failTaskOnFailedTests: true

- task: PublishBuildArtifacts@1
  condition: always()
  inputs:
    pathToPublish: 'playwright-report'
    artifactName: 'playwright-html-report'
```

Ref: *Azure DevOps - PublishTestResults task documentation*

Q36 What is the difference between Azure DevOps Classic pipelines and YAML pipelines?

ANS

Classic pipelines use a visual GUI editor and store pipeline configuration in Azure DevOps (not in source code). YAML pipelines define the pipeline as code in the repository, enabling version control, code review, and portability. YAML is the modern, recommended approach as it supports multi-stage pipelines, templates, and PR validation.

Code Example:

```
# YAML pipeline advantages over Classic:
# 1. Pipeline as code - versioned in git
# 2. PR-based changes - review pipeline changes like code
# 3. Templates - reusable pipeline components
```

```
# 4. Multi-stage - single file for entire pipeline
# 5. Portability - works in any Azure DevOps org

# Template example - reusable test steps
# templates/run-tests.yml
parameters:
  - name: environment
    type: string

steps:
  - script: npx playwright test
    env:
      BASE_URL: $[[ parameters.environment ]]

# Main pipeline using template
stages:
  - stage: Test
    jobs:
      - job: RunTests
        steps:
          - template: templates/run-tests.yml
            parameters:
              environment: 'https://staging.example.com'
```

Ref: *Azure Docs - YAML vs Classic pipelines comparison*

Q37 How do you run Playwright tests in parallel in Azure DevOps?

ANS Azure DevOps supports parallel jobs using Strategy/Matrix or by splitting tests across multiple agents. The most effective approach is to shard tests using Playwright's --shard flag, dividing the test suite across parallel agents that run simultaneously.

Code Example:

```
# azure-pipelines.yml - parallel sharding
strategy:
  matrix:
    Shard1:
      SHARD_INDEX: 1
      SHARD_TOTAL: 4
    Shard2:
      SHARD_INDEX: 2
      SHARD_TOTAL: 4
    Shard3:
      SHARD_INDEX: 3
      SHARD_TOTAL: 4
    Shard4:
      SHARD_INDEX: 4
      SHARD_TOTAL: 4

steps:
  - script: |
```

```
      npx playwright test \
        --shard=$(SHARD_INDEX)/$(SHARD_TOTAL) \
        --reporter=blob
    displayName: 'Run Playwright Shard $(SHARD_INDEX)/$(SHARD_TOTAL)'

# Merge reports after all shards complete
- script: npx playwright merge-reports ./blob-report --reporter html
```

Ref: *Playwright Docs - playwright.dev/docs/test-sharding*

Q38 How do you implement triggered pipelines and scheduled runs in Azure DevOps?

ANS CI triggers run on push/PR events. Scheduled triggers run on a cron schedule (e.g., nightly full regression). Pipeline triggers allow chaining pipelines. Resource triggers listen to changes in connected repositories or artifacts.

Code Example:

```
# azure-pipelines.yml - trigger types
# CI trigger on push
trigger:
  branches:
    include: [ main, 'release/*' ]
  paths:
    exclude: [ '*.md' ]

# PR trigger
pr:
  branches:
    include: [ main ]

# Scheduled - run full regression at 2am daily
schedules:
  - cron: '0 2 * * *'     # 2:00 AM UTC daily
    displayName: 'Nightly Full Regression'
    branches:
      include: [ main ]
    always: true           # run even if no code changes

# Pipeline trigger - run after build pipeline succeeds
resources:
  pipelines:
    - pipeline: BuildPipeline
      source: 'MyApp-Build'
      trigger: true
```

Ref: *Azure Docs - Pipeline triggers and schedules*

Q39 What are Azure DevOps service connections and how are they used in pipelines?

ANS Service connections store credentials and authentication details for external services (AWS, Azure subscriptions, Docker Hub, GitHub, SonarQube). They are managed separately from pipelines and can be restricted to specific pipelines. This centralizes secrets management and avoids embedding credentials in YAML.

Code Example:

```
# Using a service connection in azure-pipelines.yml
steps:
  # Docker Hub service connection
  - task: Docker@2
    inputs:
      containerRegistry: 'docker-hub-connection'  # service connection name
      repository: 'myorg/myapp'
      command: 'push'

  # Azure subscription service connection
  - task: AzureWebApp@1
    inputs:
      azureSubscription: 'prod-azure-subscription'
      appName: 'my-web-app'
      package: '$(Build.ArtifactStagingDirectory)/*.zip'

  # Generic REST API connection
  - task: InvokeRestAPI@1
    inputs:
      serviceConnection: 'test-reporting-api'
      method: 'POST'
      body: '{"run":"$(Build.BuildId)"}'
```

Ref: *Azure Docs - Manage service connections*

Q40 How do you integrate SonarQube/SonarCloud code analysis in Azure DevOps?

ANS SonarQube provides static code analysis, code coverage, and quality gates. In Azure DevOps, install the SonarQube extension, create a service connection, and add Prepare Analysis, Run Analysis, and Publish QualityGate tasks. Quality gates can block pipeline progression if code quality metrics fail.

Code Example:

```
# azure-pipelines.yml - SonarCloud integration
steps:
  - task: SonarCloudPrepare@1
    inputs:
      SonarCloud: 'sonarcloud-connection'
```

```
      organization: 'my-org'
      scannerMode: 'CLI'
      configMode: 'manual'
      cliProjectKey: 'my-project'
      extraProperties: |
        sonar.javascript.lcov.reportPaths=coverage/lcov.info
        sonar.testExecutionReportPaths=test-results/sonar.xml

  - script: npm run test:coverage
    displayName: 'Run Tests with Coverage'

  - task: SonarCloudAnalyze@1
    displayName: 'Run SonarCloud Analysis'

  - task: SonarCloudPublish@1
    inputs:
      pollingTimeoutSec: '300'  # wait for quality gate
```

Ref: *SonarCloud Azure DevOps integration docs - sonarcloud.io*

Q41 What is Azure Container Registry and how do you use it with Playwright tests?

ANS

Azure Container Registry (ACR) is a private Docker registry. For Playwright, you can build a custom Docker image with browsers pre-installed, push to ACR, and use it as the agent container in pipelines. This ensures consistent browser environments and faster builds (no re-downloading browsers).

Code Example:

```
# Dockerfile for Playwright tests
FROM mcr.microsoft.com/playwright:v1.40.0-focal
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .

# azure-pipelines.yml - use ACR image
pool:
  vmImage: 'ubuntu-latest'

container:
  image: myregistry.azurecr.io/playwright-runner:latest
  endpoint: acr-service-connection  # ACR service connection

steps:
  - script: npx playwright test
    displayName: 'Run E2E Tests in Container'

# Build and push image pipeline
- task: Docker@2
  inputs:
```

```
    containerRegistry: 'acr-service-connection'
    repository: 'playwright-runner'
    command: 'buildAndPush'
```

📖 **Ref:** *Azure Container Registry docs - learn.microsoft.com/en-us/azure/container-registry*

Q42 How do you manage test environments using Azure DevOps and Infrastructure as Code?

ANS

Infrastructure as Code (IaC) with tools like Terraform or Bicep provisions test environments on demand. Azure DevOps pipelines can create ephemeral environments for PR testing, run tests, then destroy the environment. This ensures isolated, reproducible test environments.

Code Example:

```
# Pipeline: PR environment lifecycle
stages:
  - stage: ProvisionEnvironment
    jobs:
      - job: TerraformApply
        steps:
          - task: TerraformTaskV4@4
            inputs:
              provider: 'azurerm'
              command: 'apply'
              environmentServiceNameAzureRM: 'azure-subscription'
              commandOptions: '-var="env_name=pr-
$(System.PullRequest.PullRequestId)"'

  - stage: RunTests
    dependsOn: ProvisionEnvironment
    jobs:
      - job: PlaywrightTests
        steps:
          - script: npx playwright test
            env:
              BASE_URL: 'https://pr-
$(System.PullRequest.PullRequestId).test.myapp.com'

  - stage: DestroyEnvironment
    dependsOn: RunTests
    condition: always()  # destroy even if tests fail
    jobs:
      - job: TerraformDestroy
        steps:
          - task: TerraformTaskV4@4
            inputs:
              command: 'destroy'
```

📖 **Ref:** *Azure DevOps + Terraform - HashiCorp Learn, Azure Bicep docs*

Q43 How do you implement retry logic and flaky test detection in Azure DevOps?

ANS

Flaky tests are non-deterministically failing tests. Playwright has built-in retries via the retries config option. In Azure DevOps, you can track flaky tests using the test analytics dashboard (Tests tab). Combine automatic retries with quarantine tags to isolate known flaky tests from blocking CI.

Code Example:

```
// playwright.config.ts - configure retries
export default defineConfig({
  retries: process.env.CI ? 2 : 0, // retry twice in CI only
  reporter: [
    ['junit', { outputFile: 'test-results/junit.xml' }]
  ]
});

// Tag flaky tests for investigation
test('flaky test @flaky', async ({ page }) => {
  // ...
});

// azure-pipelines.yml
- script: |
    # Skip flaky tests in main CI, run separately
    npx playwright test --grep-invert @flaky
  displayName: 'Run Stable Tests'

- script: |
    npx playwright test --grep @flaky --retries=3
  displayName: 'Run Flaky Tests (quarantined)'
  continueOnError: true  # don't block pipeline
```

Ref: *Playwright Docs - retries, Azure Test Analytics - flaky test detection*

Q44 What are pipeline templates in Azure DevOps and how do you create reusable ones?

ANS

Pipeline templates are reusable YAML files stored in a central repository. They define steps, jobs, or stages that can be referenced from multiple pipelines. This promotes DRY principles across teams, standardizes security practices, and enables central governance of pipeline steps.

Code Example:

```
# templates/playwright-tests.yml - reusable template
parameters:
```

```
  - name: environment
    type: string
  - name: testTags
    type: string
    default: ''
  - name: workers
    type: number
    default: 4

steps:
  - task: NodeTool@0
    inputs:
      versionSpec: '18.x'

  - script: npm ci

  - script: npx playwright install --with-deps

  - script: |
      npx playwright test \
        --grep "$[[ parameters.testTags ]]" \
        --workers $[[ parameters.workers ]]
    env:
      BASE_URL: $[[ parameters.environment ]]

# Consuming pipeline
stages:
  - stage: Test
    jobs:
      - job: E2E
        steps:
          - template: templates/playwright-tests.yml@central-templates
            parameters:
              environment: 'https://staging.example.com'
              testTags: '@smoke'
```

📖 **Ref:** *Azure Docs - Template types and usage in Azure Pipelines*

Q45 How do you use Azure DevOps Test Plans with automated tests?

ANS

Azure DevOps Test Plans provides manual testing management, but automated tests from pipelines can be associated with test cases in Test Plans. This gives traceability from requirements to automated tests and provides a unified testing dashboard showing manual + automated test status.

Code Example:

```
# Associate automated tests with Test Plans test cases
# In azure-pipelines.yml - required for Test Plans integration
- task: VSTest@2
  inputs:
    testSelector: 'testAssembly'
```

```
    runInParallel: true
    codeCoverageEnabled: true
    testRunTitle: 'E2E Regression Suite'

# OR for Playwright - use JUnit with TestSuiteMapping
- task: PublishTestResults@2
  inputs:
    testResultsFormat: 'JUnit'
    testResultsFiles: 'test-results/*.xml'
    testRunTitle: 'Playwright Tests - $(Build.BuildNumber)'
    publishRunAttachments: true

# Configure in Test Plans UI:
# - Create Test Plan > Associate pipeline
# - Map test cases to automated test methods
# - View pass/fail trends across builds
```

📖 **Ref:** *Azure DevOps - Associate automated tests with test cases*

Section 5: Best Practices & Real-World Patterns

Production-level automation patterns, anti-patterns, and architectural decisions.

Q46 What is the Testing Pyramid and how does it guide automation strategy?

ANS

The Testing Pyramid (Mike Cohn) recommends more unit tests, fewer integration tests, and fewest E2E tests. Unit tests are fast and cheap; E2E tests are slow and expensive. In automation strategy: 70% unit tests, 20% integration/API tests, 10% E2E UI tests. E2E tests should cover critical user journeys, not every possible permutation.

Code Example:

```
// Testing pyramid in practice:
// 70% - Unit tests (Jest)
describe('calculateTotal', () => {
  it('applies discount correctly', () => {
    expect(calculateTotal(100, 0.1)).toBe(90);
  });
});

// 20% - API/Integration tests (Playwright request)
test('order API returns correct total', async ({ request }) => {
  const res = await request.post('/api/orders', {
    data: { items: [{ id: 1, qty: 2 }] }
  });
  expect((await res.json()).total).toBe(59.98);
});

// 10% - E2E UI tests (Playwright browser)
```

```
test('user can complete checkout', async ({ page }) => {
  await page.goto('/products');
  await page.locator('[data-testid="product-1"]').click();
  await page.getByRole('button', { name: 'Add to Cart' }).click();
  await page.goto('/checkout');
  // complete checkout flow...
});
```

📖 **Ref:** *Mike Cohn - The Forgotten Layer of the Test Automation Pyramid*

Q47 What are the common anti-patterns in test automation and how to avoid them?

ANS Common anti-patterns: Hardcoded sleeps (use explicit waits), brittle XPath locators (use semantic locators), tests that depend on order (isolate state), testing implementation details (test user behavior), and overcrowded E2E tests (keep them focused). Each increases maintenance cost and flakiness.

Code Example:

```
// ❌ Anti-patterns
await page.waitForTimeout(3000);           // hardcoded sleep
await page.locator('//div[3]/span[1]/a');  // brittle XPath
test.only('my test', ...);                 // committed .only

// ☑ Correct approaches
await page.waitForSelector('.modal', { state: 'visible' });
await page.getByRole('link', { name: 'Login' });       // semantic
await page.getByTestId('submit-button');                // data-testid

// ❌ Tests sharing state (order-dependent)
let userId; // shared across tests - DANGEROUS
test('create user', async () => { userId = await createUser(); });
test('delete user', async () => { await deleteUser(userId); }); // depends on prev

// ☑ Isolated tests
test('create and delete user', async ({ request }) => {
  const { id } = await (await request.post('/api/users')).json();
  // ... assert creation ...
  await request.delete(`/api/users/${id}`);
});
```

📖 **Ref:** *Google Testing Blog - Just Say No to More End-to-End Tests*

Q48 How do you structure a large Playwright test project?

ANS A well-structured Playwright project separates concerns: tests in /tests, page objects in /pages, fixtures in /fixtures, utilities in /utils, test data in /test-data, and configuration in

/config. Use TypeScript for type safety. Organize tests by feature/module, not by test type.

Code Example:

```
// Recommended project structure:
// project-root/
// ├── playwright.config.ts
// ├── tests/
// │   ├── auth/
// │   │   ├── login.spec.ts
// │   │   └── register.spec.ts
// │   ├── checkout/
// │   │   └── checkout-flow.spec.ts
// ├── pages/
// │   ├── LoginPage.ts
// │   ├── DashboardPage.ts
// │   └── CheckoutPage.ts
// ├── fixtures/
// │   ├── auth.fixture.ts
// │   └── index.ts          <- export all fixtures
// ├── utils/
// │   ├── api-client.ts
// │   └── test-data-factory.ts
// ├── test-data/
// │   └── users.json
// └── .env.staging / .env.production
```

Ref: *Playwright Docs - Large scale testing best practices*

Q49 How do you implement a robust reporting strategy for test automation?

ANS

A robust reporting strategy uses multiple layers: real-time console output for local dev, JUnit XML for CI integration, HTML reports for visual review, and a dashboard (Allure, ReportPortal) for trend analysis. Include screenshots and traces on failure for fast debugging.

Code Example:

```
// playwright.config.ts - multi-reporter setup
reporter: [
  ['list'],                                    // real-time console
  ['junit', { outputFile: 'results/junit.xml' }], // CI integration
  ['html', { open: 'never', outputFolder: 'html-report' }], // visual
  ['allure-playwright'],                        // dashboard
],

// Custom reporter for Slack/Teams notification
class SlackReporter implements Reporter {
  async onEnd(result: FullResult) {
    if (result.status !== 'passed') {
      await notifySlack({
```

```
        text: `Tests FAILED: ${result.status}`,
        buildUrl: process.env.BUILD_URL
      });
    }
  }
}
```

Ref: *Playwright Docs - Reporters, Allure Framework docs*

Q50 How do you handle test data management in a CI/CD environment?

ANS

Test data strategies: Use API calls to create and teardown data within tests (preferred), use database seeding for complex scenarios, use test data factories for generating realistic data, and never share mutable test data between tests. Use unique identifiers (timestamps, UUIDs) to prevent conflicts in parallel runs.

Code Example:

```
// Test data factory pattern
import { faker } from '@faker-js/faker';

export const TestData = {
  user: (overrides = {}) => ({
    name: faker.person.fullName(),
    email: faker.internet.email(),
    password: 'TestPass123!',
    ...overrides
  }),

  product: (overrides = {}) => ({
    name: faker.commerce.productName(),
    price: parseFloat(faker.commerce.price()),
    sku: faker.string.uuid(),
    ...overrides
  })
};

// Usage in tests
test('register new user', async ({ page, request }) => {
  const user = TestData.user({ role: 'admin' });

  // Create via API (fast, reliable)
  const { id } = await (await request.post('/api/users', { data: user })).json();

  // Test UI behavior
  await loginPage.login(user.email, user.password);

  // Cleanup
  await request.delete(`/api/users/${id}`);
});
```

Ref: *Faker.js docs - fakerjs.dev, Test Data Management - Lisa Crispin*

Q51 What are webhooks and how do you test them with Playwright?

ANS

Webhooks are HTTP callbacks where a service POSTs data to your URL when events occur. Testing webhooks requires either a real endpoint, a mock server, or a service like ngrok/webhook.site. In Playwright, you can use a local Express server within the test to capture webhook payloads.

Code Example:

```
// Testing webhooks with a local server
import { createServer } from 'http';

test('payment webhook triggers order update', async ({ page, request }) => {
  let webhookPayload: any;

  // Spin up local webhook receiver
  const server = createServer((req, res) => {
    let body = '';
    req.on('data', chunk => body += chunk);
    req.on('end', () => {
      webhookPayload = JSON.parse(body);
      res.writeHead(200).end();
    });
  });
  await new Promise(r => server.listen(3001, r));

  // Trigger payment (sends webhook to our server)
  await request.post('/api/simulate-payment', {
    data: { amount: 100, webhook_url: 'http://localhost:3001/webhook' }
  });

  // Wait for webhook
  await page.waitForTimeout(2000);
  expect(webhookPayload.event).toBe('payment.completed');

  server.close();
});
```

Ref: *Playwright Docs - network, ngrok.com for webhook testing*

Q52 How do you implement accessibility testing in Playwright?

ANS

Playwright integrates with axe-core (via @axe-core/playwright) for automated accessibility testing. This checks WCAG compliance, ARIA attributes, color contrast, and keyboard navigation. Combine with manual testing for full coverage. Run as part of the CI pipeline to catch regressions.

Code Example:

```
// Install: npm install @axe-core/playwright
import { checkA11y, injectAxe } from 'axe-playwright';

test('homepage accessibility', async ({ page }) => {
  await page.goto('/');
  await injectAxe(page);

  // Check full page
  await checkA11y(page, undefined, {
    detailedReport: true,
    axeOptions: {
      runOnly: {
        type: 'tag',
        values: ['wcag2a', 'wcag2aa']  // WCAG 2.1 AA compliance
      }
    }
  });
});

// Using Playwright's built-in accessibility snapshot
const snapshot = await page.accessibility.snapshot();
expect(snapshot.children?.find(c => c.name === 'Login')).toBeTruthy();
```

Ref: *axe-core Playwright integration - github.com/abhinaba-ghosh/axe-playwright*

Q53 How do you implement performance testing with Playwright?

ANS

Playwright can capture performance metrics using the browser's **Performance API** through page.evaluate(). These metrics can include page load timings, Core Web Vitals, and resource performance.
Instead of the deprecated performance.timing, modern implementations use:

- performance.getEntriesByType("navigation")
- performance.getEntriesByName()
- PerformanceObserver for metrics like Largest Contentful Paint (LCP)

For large-scale load or stress testing, Playwright is typically combined with tools like **k6 or Artillery**, while Playwright focuses on measuring performance during real user flows.

Code Example:

```
// Capture Core Web Vitals
test('homepage performance', async ({ page }) => {
  await page.goto('/');

  const metrics = await page.evaluate(() => ({
    fcp: performance.getEntriesByName('first-contentful-paint')[0]?.startTime,
    lcp: new Promise(resolve => {
      new PerformanceObserver(list => {
        resolve(list.getEntries().at(-1)?.startTime);
```

```
      }).observe({ entryTypes: ['largest-contentful-paint'] });
    }),
    cls: 0,  // measure layout shift
    tti: performance.timing.domInteractive - performance.timing.navigationStart
  }));

  // Assert performance budgets
  expect(metrics.fcp).toBeLessThan(1500);   // FCP < 1.5s
  expect(metrics.tti).toBeLessThan(3500);   // TTI < 3.5s
});
```

Ref: *web.dev - Core Web Vitals, k6 docs - k6.io*

Q54 How do you implement a test tagging and filtering strategy?

ANS

Tags categorize tests for selective execution. Common tags: @smoke (critical path, runs in CI), @regression (full suite, nightly), @sanity (post-deploy), @flaky (quarantined), @P0/@P1 (priority). Use Playwright's --grep flag or test annotations to filter tests.

Code Example:

```
// Tagging tests with annotations
test('@smoke @P0 user can log in', async ({ page }) => {
  // critical test
});

test('@regression payment flow', async ({ page }) => {
  // full regression test
});

// playwright.config.ts - filter by environment
const testTags = process.env.TEST_TAGS || '';

// In azure-pipelines.yml
# Smoke tests on every PR
- script: npx playwright test --grep "@smoke"

# Full regression nightly
- script: npx playwright test --grep "@regression"

# Post-deploy sanity
- script: npx playwright test --grep "@sanity"

# Skip flaky in main run
- script: npx playwright test --grep-invert "@flaky"
```

Ref: *Playwright Docs - test annotations, filtering*

Q55 What is Playwright Component Testing and when should you use it?

ANS

Playwright Component Testing (CT) mounts individual UI components in real browsers without a full application context. It bridges the gap between unit tests (too shallow) and E2E tests (too heavy) for component-level testing of React/Vue/Svelte components. Use it for complex interactive components that need real browser events.

Code Example:

```
// playwright/index.html - component test config
// playwright-ct.config.ts
import { defineConfig } from '@playwright/experimental-ct-react';
export default defineConfig({
  testDir: './src',
  use: { ctPort: 3100 },
});

// Button.test.tsx - component test
import { test, expect } from '@playwright/experimental-ct-react';
import { Button } from './Button';

test('button fires onClick when enabled', async ({ mount }) => {
  let clicked = false;
  const component = await mount(
    <Button onClick={() => { clicked = true; }}>Submit</Button>
  );

  await component.click();
  expect(clicked).toBe(true);
  await expect(component).toHaveText('Submit');
});
```

Ref: *Playwright Docs - playwright.dev/docs/test-components*

Quick Reference Summary

Key topics covered across all sections:

Topic Area	Key Concepts	Interview Weight
JavaScript Core	Closures, Promises, async/await, Event Loop, ES6+	High ★★★
Playwright Basics	Auto-waiting, Locators, Contexts, POM	Critical ★★★★
Playwright Advanced	Fixtures, Tracing, Network, Visual Testing	High ★★★
Azure DevOps CI/CD	YAML Pipelines, Parallel Sharding, Environments	Critical ★★★★

Best Practices	Testing Pyramid, Anti-Patterns, Data Mgmt	Medium ★★

Total Questions: 55 | Sections: 5 | Last Updated: March 2026